

PRACTICAL : WEEK 12

Task -1 :

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 20

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d\n", i);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);

        usleep(100000);
    }

    return NULL;
}
```

```

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];

        printf("Consumed: %d\n", item);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);

        usleep(150000);
    }

    return NULL;
}

```

```

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    printf("Producer-Consumer Problem\n");
    printf("Buffer Size: %d\n", BUFFER_SIZE);
    printf("Items: %d\n\n", ITEMS);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);

    pthread_mutex_destroy(&mutex);

    printf("\nProducer and Consumer completed successfully.\n");

    return 0;
}

```

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week12
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week12$ nano producer_consumer.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week12$ gcc producer_consumer.c -o producer_consumer -pthread
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week12$ ./producer_consumer
Producer-Consumer Problem
Buffer Size: 5
Items: 20

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Produced: 5
Consumed: 4
Produced: 6
Consumed: 5
Produced: 7
Produced: 8
Consumed: 6
Produced: 9
Consumed: 7
Produced: 10
Produced: 11
Consumed: 8
Produced: 12
Consumed: 9
Produced: 13
Produced: 14
Consumed: 10
Produced: 15
Consumed: 11
Produced: 16
Consumed: 12
Produced: 17
Consumed: 13
Produced: 18
Consumed: 14
Produced: 19
Consumed: 15
Produced: 20
Consumed: 16
Consumed: 17
Consumed: 18
Consumed: 19
Consumed: 20

Producer and Consumer completed successfully.
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week12$
```

OBSERVATION:

The Producer-Consumer problem was successfully implemented using POSIX threads, counting semaphores, and a mutex. The producer added items to the shared buffer, while the consumer removed them. The empty and full semaphores prevented the buffer from becoming overfilled or empty during access, while the mutex ensured safe access to the shared buffer. The program was tested with different buffer sizes, showing that larger buffers reduced waiting and allowed smoother data flow between the producer and consumer.

Task -2 :

Design a deadlock scenario involving multiple threads competing for shared resources.

Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

CODE:

Deadlock.c

```
GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Both resources acquired\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    sleep(1);

    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Both resources acquired\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}
```

```

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}

```

Deadlock_prevent.c

```

GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Both resources acquired\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Both resources acquired\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

```

```

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Deadlock prevented using resource ordering.\n");

    return 0;
}

```

OUTPUT:

```

ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week12
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week12$ nano deadlock.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week12$ gcc deadlock.c -o deadlock -pthread
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week12$ ./deadlock
Thread 1: Locking Resource 1
Thread 2: Locking Resource 2
Thread 1: Waiting for Resource 2
Thread 2: Waiting for Resource 1
^C
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week12$ nano deadlock_prevent.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week12$ gcc deadlock_prevent.c -o deadlock_prevent -pthread
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week12$ ./deadlock_prevent
Thread 2: Locking Resource 1
Thread 2: Locking Resource 2
Thread 2: Both resources acquired
Thread 1: Locking Resource 1
Thread 1: Locking Resource 2
Thread 1: Both resources acquired
Deadlock prevented using resource ordering.
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week12$ █

```

OBSERVATION: The program successfully demonstrated a deadlock scenario between two threads competing for two shared resources. Each thread held one resource while waiting for the other, resulting in circular wait and deadlock. The four necessary conditions—mutual exclusion, hold and wait, no preemption, and circular wait—were identified. Deadlock was then prevented by enforcing a fixed resource ordering, ensuring that both threads acquired the resources in the same order.